Sharp bootstrapping thresholds for sparse-language circuit hardness and explicit obstructions

Lijie Chen, Jiatu Li, Tianqi Yang

doi:10.4230/LIPIcs.CCC.2022.23; CC BY 4.0

Editorial scope. Counted once: the Section4 sharp-bootstrapping threshold contribution, bundling Theorems4.1,4.3,4.5 and the matching unconditional obstruction/sparse-language thresholds. All original hypotheses, author proofs, continuations and supporting definitions retained. Small-error qualifiers remain unchanged. Full source-local hash and AppendixA prerequisite proofs are context, not additional counted subproblems.

Allowed standard prerequisites (editorial).

- Standard nondeterministic/parity complexity, padding, circuit encoding and imported critical-path lower bound. Full source-local hash and graph construction proofs retained as context. Imported Valiant-Vazirani parity reduction, parity closure and Fan-Li-Yang Lemma 6.4 are retained with their original citations; no recursive reproving of standard imported results. Full 43-entry bibliography.

The Minimum Circuit Size Problem with size parameter $s(n)$ (MCSP$[s(n)]$) is defined as follows: given a string x of length $N = 2^n$, determine whether there exists an n-input $s(n)$ size circuit with x as its truth table. Formally, a probabilistic circuit deciding a language L is a distribution D over circuits that outputs the wrong answer with probability at most $1/\operatorname{poly}(n)$ on any input x, i.e., for all $x \in \{0,1\}^n$ it holds that $\Pr_{D \leftarrow D}[D(x) \neq L(x)] < 1/\operatorname{poly}(n)$. We say a language L is $s(n)$-sparse if for all sufficiently large n, $L \cap \{0,1\}^n$ has size at most $s(n)$. Note that in contrast to robust classes like BPP, our result is actually sensitive to the error probability in the definition of probabilistic circuits, since the complexity overhead of error reduction is costly in the linear-size setting. For simplicity, we stick to the definition with error $1/\operatorname{poly}(n)$.

Author prerequisites (Section2)

Notation. We define $[n] \triangleq \{0, 1, \ldots, n-1\}$, $B_{n,m}$ as the set of all functions from $\mathbb{F}_2^n$ to $\mathbb{F}_2^m$ and $B_n \triangleq B_{n,1}$. The Hamming weight of $x \in \mathbb{F}_2^n$, denoted by $|x|$, is defined as the number of 1's in x; the Hamming distance $\Delta(x, y)$ of x and y from $\mathbb{F}_2^n$ is defined as the Hamming weight of the bitwise XOR of them. The concatenation of two strings x and y is denoted by $x\|y$, and the ith bit of

x (0-indexed) is denoted by x_i. Graphs are undirected by default. We assume that all functions used as parameters of our constructions are poly-logarithmic time constructible, i.e., for any function $\ell(n)$ that is used as a parameter in our results, there is a polynomial-time algorithm A such that $A(n)$ outputs the binary representation of $\ell(n)$. We use $x \leftarrow D$ to denote a random element x sampled from a distribution D, and $D(x_0) \triangleq \Pr_{x \leftarrow D}[x = x_0]$. Natural numbers are represented in binary when being fed into Turing machines or circuits; and 1^n represents the unary representation of n. We assume basic familiarity with cryptographic primitives like one-way functions and pseudorandom functions, and typical complexity classes like P, NP, and $\oplus\mathsf{P}$ (see, e.g., [3]).

2.1 Probability Theory

Let $U(S)$ be the uniform distribution supported on a set S, and $U_\ell \triangleq U(\{0,1\}^\ell)$. A family $D = \{D_n\}_{n \geq 1}$ of distributions is said to be samplable with seed length $\ell(n)$ (or $\ell(n)$-samplable) if there is a polynomial-time algorithm A such that $A(1^n, U_{\ell(n)})$ samples D_n. For every $s \in \{0,1\}^\ell$, we say $A(1^n, s)$ is the element corresponding to the seed s. The statistical distance between two distributions D_1 and D_2 over a set S, denoted by $\mathrm{SD}(D_1, D_2)$, is defined as

$$\mathrm{SD}(D_1, D_2) \triangleq \frac{1}{2} \sum_{u \in S} |D_1(u) - D_2(u)|.$$

A distribution D_2 is said to be δ-close to D_1 if their statistical distance is at most δ. Statistical distance characterizes the intractability of distinguishing two distributions by any test (even computationally unbounded): if D_1 and D_2 have statistical distance at most δ, then for any stochastic process T, $|\Pr_{u \leftarrow D_1, T}[T(u) = 1] - \Pr_{u \leftarrow D_2, T}[T(u) = 1]| \leq \delta$.

2.2 Circuit Classes

In this paper we will work with various circuit classes. In general, a circuit is an acyclic graph where each of its nodes can be an input variable, a constant $c \in \{0,1\}$, or a gate. One or more nodes are marked as output nodes (together with an index i denoting the corresponding output bit). The depth of a circuit is the number of edges on the longest path from any input variable to an output node. An n-input m-output circuit computes a function in $B_{n,m}$.

B_2 circuits. A B_2 circuit (or general circuit) contains fan-in-2 gates that can compute any binary function $f \in B_2$. The size of a B_2 circuit refers to the number of gates involved.

NC^1 circuits. An NC^1 circuit is a B_2 circuit with $O(\log n)$ depth. Note that a single-output NC^1 circuit of depth d can be converted into a formula of size $O(2^d)$.

$\mathsf{CC}^0[2]$ circuits. A $\mathsf{CC}^0[2]$ circuit is a constant-depth circuit with only XOR gates of unbounded fan-in. It is easy to see that $\mathsf{CC}^0[2]$ circuits can only compute linear functions over $\mathbb{F}_2$. The complexity of a $\mathsf{CC}^0[2]$ circuit is usually measured by the number of wires.

$\mathsf{AC}^0[m]$ **circuits.** An $\mathsf{AC}^0[m]$ circuit is a constant-depth circuit with fan-in-1 NOT gates and unbounded fan-in gates over $\{\mathrm{AND}, \mathrm{OR}, \mathrm{MOD}_m\}$, where $\mathrm{MOD}_m(x_1, \ldots, x_k) = 1$ if and only if $x_1 + x_2 + \cdots + x_k \equiv 0 \pmod{m}$. Similar to $\mathsf{CC}^0[2]$ circuits, the complexity of an $\mathsf{AC}^0[m]$ circuit is measured by the number of wires.

A probabilistic B_2 circuit (or simply a probabilistic circuit) with input size n and output size m is a distribution C_n over n-input m-output B_2 circuits. The circuit complexity of a probabilistic circuit is defined as the maximum complexity of functions in its support. A family $C = \{C_n\}_{n\geq 1}$ of n-input 1-output probabilistic circuit is said to decide a language L with error probability $\varepsilon = \varepsilon(n)$ if for sufficiently large n and all $x \in \mathbb{F}_2^n$, $\Pr_{C\leftarrow C_n}[C(x) \neq L(x)] \leq \varepsilon$.

2.3 Hash and 1-detector

Definition2.1 (Almost universal hash function). Let $m = m(n)$ and $\varepsilon = \varepsilon(n)$ be two parameters. An m-output ε-almost universal hash function is a family of distributions $H = \{H_n\}_{n\geq 1}$, where H_n is supported on $B_{n,m}$, such that $\Pr_{h\leftarrow H_n}[h(x) = h(y)] \leq \varepsilon(n)$ for all $x \neq y$ and sufficiently large n. The parameter ε is called its collision probability. It is linear if every function in the support of H_n is linear. It is said to be $s(n)$-samplable if the family H of distributions is samplable with seed length $s(n)$.

We will make heavy use of the notion of 1-detectors in [13], as parts of our almost universal hash construction.

Definition2.2 (1-detector). An m-output (randomized) 1-detector with range r and error ε is a family of distributions $D = \{D_n\}_{n\geq 1}$, where D_n is supported on linear functions in $B_{n,m}$, such that for sufficiently large n, $\Pr_{L\leftarrow D_n}[L(x) \neq 0] \leq \varepsilon(n)$ for all $x \in \mathbb{F}_2^n$ satisfying $0 < |x| \leq r$. It is said to be $s(n)$-samplable if the family D of distributions is samplable with seed length $s(n)$.

We emphasize that the definition requires the functions in the family of distributions to be linear. This ensure that, for any randomized 1-detector D_n with range r and error ε, and any $x_1 \neq x_2$ with Hamming distance at most r, it holds that $\Pr_{L\leftarrow D_n}[L(x_1) \neq L(x_2)] \leq \varepsilon(n)$. The shrinkage of a hash function (or 1-detector) is defined as the input length n divided by the output length m (e.g., poly-logarithmic shrinkage means $m = n/\operatorname{polylog}(n)$).

2.4 ε-biased set and k-wise independence

To derandomize the low-complexity hash function in [13], we need several standard derandomization tools including strongly explicit ε-biased sets, k-wise independent distributions, and k-wise independent permutations.

Definition2.3 (ε-biased set). For any $n \geq 1$ and $\varepsilon \in (0, 1/2)$, a set $S \subseteq \mathbb{F}_2^n$ is said to be ε-biased if for all non-zero $v \in \mathbb{F}_2^n$, $\Pr_{w\leftarrow S}[\langle w, v\rangle = 0] \in [1/2 - \varepsilon, 1/2 + \varepsilon]$.

Theorem2.4 (Alon, Goldreich, Håstad, and Peralta [1]). For any constant $\varepsilon \in (0, 1/2)$, there is a family $\{S_n \subseteq \mathbb{F}_2^n\}_{n\geq 1}$ of ε-biased sets such

that $|S_n| = \widetilde{O}(n^2)$. Moreover, there is an algorithm $A(n, i, j)$ running in time $\mathrm{poly}(\log n)$ that computes the jth bit of the ith vector in S_n.

Definition2.5. A k-wise ε-dependent distribution with length n and alphabet size p is a distribution D over $[p]^n$ such that for all $0 \le i_1 < i_2 < \cdots < i_k < n$, the distribution over $[p]^k$ obtained by restricting D to the i_1th, i_2th, ..., i_kth coordinates is ε-close to the uniformly distribution over $[p]^k$. It is said to be a k-wise independent distribution if $\varepsilon = 0$.

Theorem2.6 (Alon, Goldreich, Håstad, and Peralta [1]). Let $\varepsilon = \varepsilon(n) > 0$ be a parameter. There is an algorithm A such that $A(1^n, k, U_r)$ runs in time $\mathrm{poly}(n, \log(1/\varepsilon))$ and samples a k-wise ε-dependent distribution with length n and alphabet size 2 for every $1 \le k \le n$, where $r = O(k + \log\log n + \log(1/\varepsilon))$. Moreover, there is an algorithm B such that $B(n, k, v, i)$ runs in $\mathrm{poly}(\log n, k, \log(1/\varepsilon))$ time and computes the ith coordinate of the string corresponding to the seed v.

Definition2.7. Let $n \ge 1$ be the number of elements, and P_n be the set of all permutations over $[n]$. A k-wise ε-dependent permutation is a distribution D over P_n such that for all $0 \le i_1 < i_2 < \cdots < i_k < n$, the distribution of $(f(i_1), f(i_2), \ldots, f(i_k))$ with $f \leftarrow D$ is ε-close to the uniform distribution over $\{(j_1, j_2, \ldots, j_k) \mid j_1, \ldots, j_k \in [n] \text{ are pairwisely distinct}\}$.

Theorem2.8 (Kaplan, Naor, and Reingold [25], Theorem5.9). Let n be a power of 2, $2 \le k \le n$, and $\varepsilon > 2^{-n}$. There is an n-element k-wise ε-dependent permutation that is samplable with seed length $O(k \log n + \log(1/\varepsilon))$. Moreover, there is an algorithm $A(n, v, i)$ that runs in $\mathrm{poly}(\log n, k, \log(1/\varepsilon))$ time and outputs $\rho_v(i)$ for the permutation ρ_v corresponding to the seed v.

2.5 Expander Graphs

We will need strongly explicit expander graphs in our proofs. We first recall the definition of expander graphs.

Definition2.9. An n-vertex d-regular graph G is called an (n, d, λ) expander graph (or (n, d, λ)-graph) if $\lambda_2(G) \le \lambda$, where $\lambda_2(G)$ denotes the second largest eigenvalue of normalized adjacency matrix (i.e., adjacency matrix divided by d) of G. A family of graphs $\{G_n\}_{n \ge 1}$ is an expander graph family if there exists constants d and $\lambda < 1$ such that for all sufficiently large n, G_n is an (n, d, λ)-graph.

We will make use of the following construction of strongly explicit expander graphs.

Theorem2.10 (Strongly explicit expander; see [3, Theorem21.19]). There exists an expander graph family $\{G_n\}_{n \ge 1}$ and an algorithm $A(n, v, i)$ that runs in polylog(n) time (i.e., polynomial in input length) and outputs the ith neighbor of the node v in G_n, where $v \in [n]$ and $i \in [d]$.

Performing random walk on expanders is a standard derandomization technique. Consider the task to find a good element from n elements in which there is a constant fraction of them being good. A trivial approach is to sample ℓ

independently random elements, which has $\exp(-\Omega(\ell))$ error probability but requires $\ell \log n$ random bits. By applying the following lemma, we can reduce the randomness complexity to $O(\log n + \ell)$ while keeping the error probability to be exponentially small.

Lemma2.11 (Expander walk; see [3, Theorem21.12]). Let G be an (n, d, λ)-graph and S be a subset of vertices of size at most βn for some $\beta \in (0, 1)$. Let $X_1, X_2, \ldots, X_k$ be random variables denoting a random walk in G (where X_1 is uniformly chosen), then

$$\Pr[\forall 1 \leq i \leq k, X_i \in S] \leq ((1-\lambda)\sqrt{\beta} + \lambda)^{k-1}.$$

2.6 Graph with large girth

The girth of an undirected graph is the length of the minimum cycle in it. We need the following construction of strongly explicit graphs with large girth.

Theorem2.12 (Adapted from [27]; see AppendixA). Let $r = r(n) = n^{o(1)}$ be a parameter. For every sufficiently large n, there exists an $m = \Theta(n/r)$ and a regular graph $G_{m,n}$ with m vertices, n edges, and girth $\Omega(\log n / \log r)$. Moreover, there exists a polylog(n)-time algorithm $A(n, i)$ for $i \in [n]$ that outputs the indices of the two endpoints of the ith edge in $G_{m,n}$, and a polylog(n)-time algorithm $B(n, i, j)$ for $i \in [m]$ that outputs the jth edge attaching to the ith vertex.

3 Randomness-efficient low-complexity hash functions

In this section, we present constructions of randomness-efficient low-complexity hash functions with various parameters and properties. In Section3.1, we show almost universal hash functions can be constructed from 1-detectors (Lemma3.2). In Section3.2 we give a derandomized version of construction of 1-detectors from [13] based on high-girth graphs (Lemma3.3), and use that to construct a low-complexity hash function with poly-logarithmic seed-length and $n/\operatorname{polylog}(n)$ output length (Theorem3.4). In Section3.3, we show how to reduce the output length from $n/\operatorname{polylog}(n)$ to polylog(n), by composing almost universal hash families. Finally, in Section3.4 and3.5, we establish the uniformity and explicitness of our constructions, which are essential for our applications to hardness magnification and PRF constructions.

Notation. In this section, for a construction H with parameters a, b, c, we will use $H_{\hat{a},\hat{b},\hat{c}}$ to denote the specific construction H with the parameters specified to $\hat{a}, \hat{b}$ and $\hat{c}$. When the parameters are obvious from the context, we often omit the subscripts and simply write it as H.

3.1 General construction from 1-detectors

We first give a general construction of almost universal hash functions from 1-detectors, using the following sampling procedure.

Lemma3.1. For all integers n, b and r such that $b \leq n$ and $\max\{10n/b, 10\log\log n\} \leq r \leq n$, there is a distribution $D^{\mathrm{samp}}_{n,b,r}$ supported on $[n]^b$ samplable by $O(r\log(n/b))$ bits, such that the following conditions hold:

1. (Hitting Condition). For all $x \in \mathbb{F}_2^n$ with $|x| \geq r$, it holds that
$$\Pr_{(w_0,\ldots,w_{b-1})\leftarrow D^{\mathrm{samp}}_{n,b,r}}\left[\bigwedge_{j\in[b]}[x_{w_j}=0]\right] \leq 2\exp\left[-\frac{br}{2n}\right]. \tag{1}$$
2. (Ordering) For every $(w_0,\ldots,w_{b-1}) \in \mathrm{supp}(D^{\mathrm{samp}}_{n,b,r})$, it holds that $w_0 < w_1 < \cdots < w_{b-1}$.
3. (Explicitness) There are algorithms $A^{\mathrm{samp}}_{n,b,r}(v,i)$ and $B^{\mathrm{samp}}_{n,b,r}(v,j)$ running in $\mathrm{poly}(\log n, r)$ time such that
$$A^{\mathrm{samp}}_{n,b,r}(v,i) \text{ outputs } \begin{cases} j & \text{if } w_j = i \text{ for some } j\in[b],\\ \perp & \text{otherwise;}\end{cases}$$
$$B^{\mathrm{samp}}_{n,b,r}(v,j) \text{ outputs } w_j,$$
where $(w_0,\ldots,w_{b-1}) \in \mathrm{supp}(D^{\mathrm{samp}}_{n,b,r})$ is the vector corresponding to the seed v.

Proof. We firstly describe a sampling procedure that satisfies Equation(1) but has a long seed length and then reduce the seed length to $O(r\log(n/b))$ bits using the explicit k-wise ε-dependent distribution from Theorem2.6. The sampling procedure is simple: we first partition $[n]$ into b consecutive groups $g_0, g_1, \ldots, g_{b-1}$ of size either $\lfloor n/b\rfloor$ or $\lceil n/b\rceil$; we then uniformly choose an $i_j \in g_j$ for each $j \in [b]$; finally we output the tuple $(i_0, i_1, \ldots, i_{b-1})$. For any $x \in \mathbb{F}_2^n$ with Hamming weight at least r, for some $\ell \leq r$, there are ℓ groups such that there are at least r 1-indices (i.e., the corresponding bits of x are 1) in these groups. Assume that there are a_j 1-indices in the jth among these ℓ groups. The probability that none of the 1-indices is sampled is at most
$$\begin{aligned}\prod_{j=1}^{\ell}\left[1-\frac{a_j b}{2n}\right] &\leq \exp\left(\sum_{j=1}^{\ell}\log\left[1-\frac{a_j b}{2n}\right]\right)\\ &\leq \exp\left(-\sum_{j=1}^{\ell}\frac{a_j b}{2n}\right) \leq \exp\left[-\frac{br}{2n}\right].\end{aligned} \tag{2}$$

A trivial implementation of the sampling procedure above needs seed length $O(b\log(n/b))$: for each group, we sample a random binary string of length $t \triangleq \lceil\log(n/b)\rceil$ indicating the index we want to choose. Therefore, we would need to sample bt bits in total. The observation here is that the argument in

(2) only involves $\ell \leq r$ groups. Hence, the analysis still works if we sample those required bt bits from an rt-wise $\exp(-br/(2n))$-dependent distribution with alphabet 2. Note that this incurs an additional error of $\exp(-br/(2n))$. By Theorem2.6, the distribution is samplable with seed length

$$O(rt + \log\log(bt) + br/(2n)) = O(r\log(n/b)),$$

and the algorithms $A^{\mathrm{samp}}_{n,b,r}$ and $B^{\mathrm{samp}}_{n,b,r}$ can be constructed straightforwardly from the algorithm B in Theorem2.6.[1] □

Now we are ready to give the construction of our hash function from 1-detectors. Recall that their definitions are given in Section2.3.

Construction1 (Hash function H^L from 1-detectors L). Let n be the input length and $b(n) = o(n)$ be a parameter. Given a randomized 1-detector $L = \{L_n\}_{n\geq 1}$ with range $\hat{r}$, we construct a hash function $H^L_b = \{H_n\}_{n\geq 1}$ as follows.

- Let D^{samp} be the distribution in Lemma3.1 with parameters n, b and $r = \hat{r}$. For each L in the support of L and each $D = (j_0, \ldots, j_{b-1})$ in the support of D^{samp}, we define a function $h_{L,D}$ as

$$h_{L,D}(x) \triangleq L(x)\|x_{j_0}\|x_{j_1}\|\cdots\|x_{j_{b-1}}.$$

- H_n is then defined to be the distribution generated as follows: sample $L \leftarrow L_n$ and $D \leftarrow D^{\mathrm{samp}}$, and then output $h_{L,D}$.

We show that Construction1 indeed gives almost universal hash functions.

Lemma3.2 (H^L is a linear almost universal hash). Let $L = \{L_n\}_{n\geq 1}$ be an $\hat{m}$-output $\hat{s}$-samplable randomized 1-detector with error $\hat{\varepsilon}$ and range $\hat{r}$, such that $\hat{m} = o(n)$ and $\hat{r} = \omega(\log n)$. Let $b = b(n) = o(n)$ be a parameter and $r = r(n) = \hat{r}(n)$. The followings hold for H^L_b:

1. H^L_b has output length $m = \hat{m} + b$.
2. H^L_b is $(\hat{s} + \hat{r}\log(n/b))$-samplable.
3. H^L_b is a linear ε-almost universal hash function, where $\varepsilon \triangleq \hat{\varepsilon} + \exp(-\Omega(br/n))$.

Proof. The first two items follow directly from the definition of H^L from Construction1. So we will only establish the last item. We need to show that for any two inputs $x_1 \neq x_2$ from $\{0,1\}^n$, their hash values collide with probability at most ε. Consider the following two cases depending on whether the Hamming distance between x_1 and x_2 is small or large:

- If $0 < \Delta(x_1, x_2) \leq r$, then by the definition of randomized 1-detector (Definition2.2), $L(x_1)$ equals to $L(x_2)$ with probability at most $\hat{\varepsilon}$ for $L \leftarrow L_n$ (note that L is a linear function over $\mathbb{F}_2$). Since the hash values contain the output of the 1-detector as the first $\hat{m}$ bits, the hash values collide with probability at most $\hat{\varepsilon}$.

[1] To compute $A^{\mathrm{samp}}_{n,b,r}(v,i)$ for some i belonging to the group g_a, we only need to check if i is the selected element of the group g_a, and then output a if the answer is yes, and output $\perp$ otherwise.

- If $\Delta(x_1, x_2) > r$, then $y \triangleq x_1 \oplus x_2$ has Hamming weight at least r. By Lemma3.1, a random $D \leftarrow D^{\mathrm{samp}}_{n,b,r}$ fails to contain all 1-indices of y with probability at most $\exp(-\Omega(br/n))$, which means that the last b bits of the hash values of x_1 and x_2 collide with probability at most $\exp(-\Omega(br/n))$.

It follows immediately that the collision probability is at most $\varepsilon = \hat{\varepsilon} + \exp(-\Omega(br/n))$. □

3.2 Randomness-efficient low-complexity 1-detectors

Now we will present the construction of randomness-efficient low-complexity 1-detectors. Applying Lemma3.2, this construction yields a hash function with inverse-super-polynomial collision probability, short seed length, and moderately large shrinkage.

Construction2 (Construction of the 1-detector L^{hg} from high-girth graphs). Let $k = k(n) = n^{o(1)}$ be the shrinkage parameter, $\log n \le r(n) \le n/2$ be the range. Let $\gamma = \gamma(n) \triangleq \log^2 n / \log k$. $L^{\mathrm{hg}}_{k,r} = \{L_n\}_{n \ge 1}$ is constructed as follows.

- Let n' be the smallest power of two that is larger than n. Let $G_{m,n'}$ be the regular graph with $m = \Theta(n'/k)$ vertices, n' edges, and girth $g = \Theta(\log n / \log k)$ from Theorem2.12. Let $D = \{D_{n'}\}$ be the explicit family of r-wise $2^{-\gamma}$-dependent permutation in Theorem2.8. Let $\Gamma(i)$ be the set of indices of edges incident to the ith vertex in $G_{m,n'}$. For each permutation $\sigma \in \mathrm{supp}(D_{n'})$, we define a function $L_\sigma : \{0,1\}^n \to \{0,1\}^m$ as
$$L_\sigma(x)_i \triangleq \bigoplus_{j \in [n] \text{ s.t. } \sigma(j) \in \Gamma(i)} x_j \qquad \forall i \in [m].$$
- L_n is then defined to be the distribution generated as follows: sample $\sigma \leftarrow D_{n'}$ and then output L_σ.

Lemma3.3 (L^{hg} is a 1-detector). Let $k = k(n) = n^{o(1)}$ be the shrinkage parameter, $\log n \le r(n) \le n/2$ be the range. The followings hold for $L^{\mathrm{hg}}_{k,r} = \{L_n\}_{n \ge 1}$:

1. $L^{\mathrm{hg}}_{k,r}$ has output size $\Theta(n/k)$ and seed length $O(r \log n)$.
2. Every $L \in L_n$ can be computed by depth-1 $\mathsf{CC}^0[2]$ circuits of wire complexity at most $2n$, or B_2 circuits of size $2n$ and depth $\log k + O(1)$.
3. $L^{\mathrm{hg}}_{k,r}$ is a randomized 1-detector with range r and error $\exp(-\Omega(\log^2 n / \log k))$.

Proof. Recall that $\gamma = \log^2 n / \log k$ from Construction2. For the first item, the output size follows directly from the definition of $L^{\mathrm{hg}}_{k,r}$. The seed length of $L^{\mathrm{hg}}_{k,r}$ equals the seed length of the required r-wise $2^{-\gamma}$-dependent permutation, which is $O(r \log n + \gamma) = O(r \log n)$ from Theorem2.8 (note that $r \ge \log n$). The second item follows directly from the definition of L_σ in Construction2 together with the fact that $G_{m,n'}$ has maximum degree $O(k)$ (since it is regular). In the rest

of the proof we establish the third item. We need to show that for any non-zero $x \in \mathbb{F}_2^n$, the probability that $L(x) = 0$ with $L \leftarrow L_n$ is at most $\exp(-\Omega(\gamma))$. For any input x with Hamming weight $0 < |x| < g$ (where g is the girth of $G_{m,n}$), we must have $L_\sigma(x) \neq 0$ for all $\sigma \in D_{n'}$, since otherwise we can extract from the edges $\{\sigma(i) \mid x_i = 1 \wedge i \in [n]\}$ a cycle of size length less than g. Now we consider the case when $g \leq |x| \leq r$. Let $i_1, i_2, \ldots, i_{|x|}$ be the indices corresponding to the 1's in x (i.e., $x_{i_j} = 1$ for all j). Since our construction is linear, only these bits can influence the output. Also, by the r-wise $2^{-\gamma}$-dependence of $D_{n'}$, we have

$$\begin{aligned}
\Pr_{\sigma \leftarrow D_{n'}}[L_\sigma(x) = 0] &= \Pr_{\sigma \leftarrow D_{n'}}\left[\bigwedge_{i' \in [m]} [|\{i_j \mid \sigma(i_j) \in \Gamma(i')\}| \text{ is even}]\right] \\
&\leq \Pr_{S \subseteq [n'],\, |S| = |x|}\left[\bigwedge_{i' \in [m]} [|S \cap \Gamma(i')| \text{ is even}]\right] + 2^{-\gamma} \\
&= \mathbb{E}_{\substack{S \subseteq [n'] \\ |S| = |x| - \alpha}}\left[\Pr_{\substack{S' \subseteq [n'] \setminus S \\ |S'| = \alpha}}\left[\bigwedge_{i' \in [m]} [|(S \cup S') \cap \Gamma(i')| \text{ is even}]\right]\right] + 2^{-\gamma},
\end{aligned}$$

where $\alpha \triangleq \lceil g/3 \rceil$. The final observation is that for any fixed S, there should be at most one S' satisfying the condition in the summation. Indeed, if two sets S'_1 and S'_2 satisfy the condition at the same time, then the symmetric difference of them contains a cycle in $G_{m,n'}$ of length $2\alpha < g$ for sufficiently large n, contracting the fact that $G_{m,n'}$ has girth g. In particular, it means for every fixed S, we have

$$\Pr_{\substack{S' \subseteq [n'] \setminus S \\ |S'| = \alpha}}\left[\bigwedge_{i' \in [m]} [|(S \cup S') \cap \Gamma(i')| \text{ is even}]\right] \leq \frac{1}{\binom{n' - |x| + \alpha}{\alpha}}.$$

Since $\alpha = \lceil g/3 \rceil = \Theta(\log n / \log k)$, putting everything together, for sufficiently large n we have

$$\Pr_{\sigma \leftarrow D_n}[L_\sigma(x) = 0] \leq \frac{1}{\binom{n' - |x| + \alpha}{\alpha}} + 2^{-\gamma} = \exp(-\Omega(\gamma)).$$

□

Plugging L^{hg} into Construction1 with appropriately chosen parameters, we immediately obtain a linear hash H^{md} with poly-logarithmic seed length and shrinkage as follows. (Here md stands for moderate, meaning that the hash has moderate (poly-logarithmic) shrinkage.)

Construction3 (Construction of hash H^{md} with poly-logarithmic shrinkage). Let $\beta \in (0, 2]$ be a constant and $\omega(\log n) \leq \delta(n) \leq O(\log^2 n / \log\log n)$ be the error parameter. $H^{\mathrm{md}}_{\beta,\delta} = \{H_n\}_{n \geq 1}$ is constructed as follows.

- Setting $k = \log^\beta n$ and $r = \delta \log^\beta n$, $L^{\mathrm{hg}} = L^{\mathrm{hg}}_{k,r}$ (from Construction2) is an $O(\delta \log^{\beta+1} n)$-samplable $\Theta(n/\log^\beta n)$-output randomized 1-detector with range r and error $\exp(-\Omega(\delta))$.

- Setting $b = n/\log^{\beta} n$, H^{md} is now defined to be $H_b^{L^{\mathrm{hg}}}$ (from Construction1).

The following theorem follows directly from the Construction3, Lemma3.2, and Lemma3.3.

Theorem3.4 (Properties of the intermediate hash construction H^{md}). Let $\beta \in (0, 2]$ be the shrinkage parameter and $\omega(\log n) \le \delta(n) \le O(\log^2 n/\log\log n)$ be the error parameter. The followings hold for $H^{\mathrm{md}}_{\beta,\delta} = \{H_n\}_{n\ge 1}$:

1. $H^{\mathrm{md}}_{\beta,\delta}$ is a linear $O(\delta \log^{\beta+1} n)$-samplable $\Theta(n/\log^{\beta} n)$-output $\exp(-\Omega(\delta))$-almost universal hash function.
2. Every $H \in H_n$ can be computed either by a depth-1 $\mathsf{CC}^0[2]$ circuit of wire complexity $2n$, or by a B_2 circuit of size $2n$ and depth $\beta \log\log n + O(1)$.

3.3 Shrinkage reduction of hash function

Now we reduce the output length of the hash function H^{md} in Construction3 from $n/\operatorname{polylog}(n)$ to $\operatorname{polylog}(n)$ with little overhead in its circuit complexity and seed length, which is crucial for our applications. The idea is simple: we composite it with a hash with large shrinkage, short seed length, and relatively larger circuit complexity. In particular, we can use the following hash construction H^{expw} of Chen, Jin, and Williams [9]. (Here expw stands for expander walk.)

Construction4 (Hash function H^{expw} from expander walk [9]). Let $\ell = \ell(n)$ be the output length. $H^{\mathrm{expw}}_{\ell} = \{H_n\}_{\ge n}$ is constructed as follows.

- Let $\{S_n\}_{n\ge 1}$ be a family of 0.1-biased set from Theorem2.4 and $\{G_n\}_{n\ge 1}$ be the family of strongly explicit expanders from Theorem2.10. Assume that $S_n = \{w_0, w_1, \ldots, w_{t-1}\}$ for $t = \widetilde{O}(n^2)$.
- For each walk $v = (v_0, v_1, \ldots, v_{\ell-1}) \in [t]^{\ell}$ of length ℓ on G_t, we define a hash function $h_v : \{0,1\}^n \to \{0,1\}^{\ell}$ as

$$h_v(x) \triangleq \langle w_{v_0}, x\rangle \| \langle w_{v_1}, x\rangle \| \cdots \| \langle w_{v_{\ell-1}}, x\rangle,$$

  where $\langle a, b\rangle$ denotes the inner product over $\mathbb{F}_2$.
- H_n is then defined to be the distribution generated as follows: samples a random walk v of length ℓ on G_t uniformly at random, then outputs h_v.

Lemma3.5 (Properties of the hash construction H^{expw} from [9]). Let $\ell = \ell(n)$ be the output length. The followings hold for $H^{\mathrm{expw}}_{\ell} = \{H_n\}_{n\ge 1}$:

1. Every $H \in \operatorname{supp}(H_n)$ can implemented by a depth-1 $\mathsf{CC}^0[2]$ circuit of wire complexity $n\ell$ or a B_2 circuit of size $n\ell$ and depth $\log n + O(1)$.
2. H^{expw}_{ℓ} is a linear $O(\log n + \ell)$-samplable ℓ-output $\exp(-\Omega(\ell))$-almost universal hash function.

Proof. The first item follows directly from the definition of h_v from Construction4. In the following we establish the second item. The linearity, seed length, and output length are straightforward to verify, thus we only analyze the collision probability. Since the hash function is linear, we only need to show that for any non-zero $x \in \mathbb{F}_2^n$, $\Pr_{h \leftarrow H_n}[h(x) = 0] \leq \exp(-\Omega(\ell))$. By Theorem2.4, we know that for any non-zero x, at least 0.4 fraction of vectors $w \in S_n$ satisfies $\langle w, x \rangle = 1$. Then by Theorem2.11, a random walk of length ℓ will find one of such w with probability $1-\exp(-\Omega(\ell))$. This immediately implies that $h(x) \neq 0$ with probability at most $\exp(-\Omega(\ell))$ for $h \leftarrow H_n$. □

Next we formally define the composition of two hash function families.

Definition3.6 (Composition of hash families). Let $F = \{F_n\}_{n \geq 1}$ and $G = \{G_n\}_{n \geq 1}$ be two families of hash functions. The composition of F and G is defined as $F \circ G = \{(F \circ G)_n\}_{n \geq 1}$, where $(F \circ G)_n$ is the following distribution: let $m = m(n)$ be the output length of G, we sample $f \leftarrow F_m$ and $g \leftarrow G_n$, and then output $f \circ g$.

The following proposition is crucial for the analysis of our final hash construction.

Proposition3.7. The composition $H' \circ H$ of an ε_1-almost universal hash function H' and an ε_2-almost universal hash function H is an $(\varepsilon_2(n)+\varepsilon_1(m(n)))$-almost universal hash function, where $m(n)$ is the output length of H. Moreover, $H' \circ H$ is linear if both of H' and H are linear.

The following is a simple corollary of Proposition3.7.

Corollary3.8. For any $t \geq 2$, non-increasing $\varepsilon = \varepsilon(n)$, and output length parameter $\ell(n) \leq n$, the tth order composition of an ℓ-output ε-almost universal hash H with itself, denoted by $H^{\circ t}$, is also a hash function with collision probability $t \cdot \varepsilon(\hat{\ell})$, where $\hat{\ell} = \ell \circ \ell \circ \cdots \circ \ell(n)$ ($t-1$ times) is the input of the outer-most hash. Moreover, $H^{\circ t}$ is linear if H is linear.

We are now ready to specify our final hash construction H^{final} with $\text{polylog}(n)$ output length.

Construction5 (Construction of H^{final} with $\text{polylog}(n)$ output length). Let $\beta \in (0, 2]$ be a parameter and $\omega(\log n) \leq \ell(n) \leq O(\log^2 n / \log\log n)$ be a non-decreasing function. We define

$$H^{\text{final}}_{\beta,\ell} \triangleq H^{\text{expw}}_{\ell} \circ (H^{\text{md}}_{\beta,\ell})^{\circ \lceil 2/\beta \rceil}.$$

We analyze the properties of H^{final} constructed above by combining the composition proposition (Proposition3.7 and Corollary3.8) together with properties of H^{expw} (Lemma3.5) and H^{md} (Theorem3.4).

Theorem3.9 (Properties of the final hash construction H^{final}). Let $\beta \in (0, 2]$ be a constant and $\omega(\log n) \leq \ell(n) \leq O(\log^2 n / \log\log n)$ be a non-decreasing function. The followings hold for $H^{\text{final}}_{\beta,\ell} = \{H_n\}_{n \geq 1}$:

1. $H^{\text{final}}_{\beta,\ell}$ is a linear $O(\ell \log^{\beta+1} n)$-samplable $\Theta(\hat{\ell})$-output $\exp(-\Omega(\hat{\ell}))$-almost universal hash function, where $\hat{\ell} = \ell(\Theta(n/\log^{\beta \cdot \lceil 2/\beta \rceil} n))$.

2. Every $H \in H_n$ can be computed by depth-$(1 + \lceil 2/\beta \rceil)$ $\mathsf{CC}^0[2]$ circuits of wire complexity $2n + O(n\hat{\ell}/\log^2 n)$, or by B_2 circuits of size $2n + O(n\hat{\ell}/\log^2 n)$ and depth $\log n + O(1)$.

Proof. Let $H^{\mathrm{md}} = H^{\mathrm{md}}_{\beta,\ell}$ and $H^{\mathrm{expw}} = H^{\mathrm{expw}}_{\ell}$ be the hash functions described in Construction5. We also use H^{md}_n and H^{expw}_n to denote the distributions over n-input hash functions in H^{md} and H^{expw}, respectively.

H^{final} is almost universal with $\mathrm{polylog}(n)$ **output length.** Let $\hat{m}$ be the output length of $(H^{\mathrm{md}})^{\circ\lceil 2/\beta \rceil}$, and $\hat{\ell} \triangleq \ell(\hat{m}) = \ell(\Theta(n/\log^{\beta\cdot\lceil 2/\beta\rceil} n))$. By a simple induction and Theorem3.4 we know that for any $1 \le d \le \lceil 2/\beta \rceil$, $(H^{\mathrm{md}})^{\circ d}$ is a linear $O(\ell \log^{\beta+1} n)$-samplable $\Theta(n/\log^{\beta d} n)$-output $\exp(-\Omega(\hat{\ell}))$-almost universal hash function. In particular, when $d = \lceil 2/\beta \rceil$, we know that $(H^{\mathrm{md}})^{\circ\lceil 2/\beta\rceil}$ is a linear $O(\ell \log^{\beta+1} n)$-samplable $\Theta(n/\log^{\beta\cdot\lceil 2/\beta\rceil} n)$-output $\exp(-\Omega(\hat{\ell}))$-almost universal hash function. Now apply Proposition3.7 and Lemma3.5, we know that $H^{\mathrm{final}} = H^{\mathrm{expw}} \circ (H^{\mathrm{md}})^{\circ\lceil 2/\beta\rceil}$ is a linear $O(\ell \log^{\beta+1} n)$-samplable $\Theta(\hat{\ell})$-output $\exp(-\Omega(\hat{\ell}))$-almost universal hash function.

Complexity with $\mathsf{CC}^0[2]$ circuits. By Theorem3.4, every $H \in \mathrm{supp}(H^{\mathrm{md}}_n)$ can be computed by a depth-1 $\mathsf{CC}^0[2]$ circuit with wire complexity $2n$. Also, by Lemma3.5, every $H \in \mathrm{supp}(H^{\mathrm{expw}}_n)$ can be computed by a depth-1 $\mathsf{CC}^0[2]$ circuit with wire complexity $n\ell$. Since the output length of $(H^{\mathrm{md}})^{\circ d}$ is $\Theta(n/\log^{\beta d} n)$, the total wire complexity of a hash function from the support of $H^{\mathrm{expw}} \circ (H^{\mathrm{md}})^{\circ\lceil 2/\beta\rceil}$ can be bounded by

$$2n + O\left(\sum_{d=1}^{\lceil 2/\beta\rceil - 1} \frac{n}{\log^{\beta d} n}\right) + \frac{n}{\log^{\beta\cdot\lceil 2/\beta\rceil} n} \cdot \hat{\ell} = 2n + O\left(\frac{n\hat{\ell}}{\log^2 n}\right).$$

The depth complexity can be verified straightforwardly.

Complexity with B_2 circuits. We only analyze the depth since the analysis of the size complexity is similar to the analysis above. By Theorem3.4 and Lemma3.5, every $H \in \mathrm{supp}(H^{\mathrm{md}}_n)$ can be computed by a B_2 circuit with depth $\beta \log\log n + O(1)$, and every $H \in \mathrm{supp}(H^{\mathrm{expw}}_n)$ can be computed by a B_2 circuit with depth $\log n + O(1)$. Therefore the total depth of a hash function from the support of $H^{\mathrm{expw}} \circ (H^{\mathrm{md}})^{\circ\lceil 2/\beta\rceil}$ is at most

$$\sum_{d=0}^{\lceil 2/\beta\rceil - 1} \left[\beta \log\log\left(\Theta\left(\frac{n}{\log^{\beta d} n}\right)\right) + O(1)\right] + \log\left(\Theta\left(\frac{n}{\log^{\beta\cdot\lceil 2/\beta\rceil} n}\right)\right) = \log n + O(1).$$

□

3.4 Explicitness of our construction

Apart from the seed length, shrinkage and collision probability, the explicitness of the a hash function is also critical for many applications. That is, given a seed v (which is usually much shorter than the input, say $|v| = \mathrm{polylog}(n)$),

whether we can obtain information about the hash function corresponding to the seed v efficiently. In this section we show that our hash constructions are indeed explicit in a strong sense, which is crucial for our application to hardness magnification.

Definition3.10 (Locally explicit hash function). A family of polylog(n)-samplable distributions over m-output linear functions (e.g., linear hash functions or 1-detectors) $H = \{H_n\}_{n\geq 1}$ is said to be locally explicit if each input bit only influences polylog(n) output bits, and there exists an algorithm $A(n, v, i)$ running in polylog(n) time that returns the list of output bits influenced by the ith input bit in the hash function corresponding to the seed v.

Note that any linear function can be realized as $x \mapsto Mx$ for some transformation matrix M over $\mathbb{F}_2$. Clearly, a locally explicit linear hash has a sparse transformation matrix, and we can efficiently list all 1-entries in any column of it. By performing a sparse matrix multiplication, we can immediately obtain the following proposition.

Proposition3.11. The composition of two locally explicit hash families is locally explicit.

Now we verify that our constructions of the randomness-efficient low-complexity hash functions are indeed locally explicit.

Theorem3.12. Let $k = k(n) = n^{o(1)}$ be the shrinkage parameter and $\log n \leq r(n) \leq n/2$ be the range. The 1-detector $L^{\mathrm{hg}}_{k,r}$ in Construction2 is locally explicit.

Proof. Recall that the 1-detector in Construction2 consists of two components: a depth-1 $\mathsf{CC}^0[2]$ circuit whose topology is determined by the high-girth graph from Theorem2.12, and a random permutation of the input bits according to the k-wise almost independent permutation from Theorem2.8. Given the seed v of the 1-detector (i.e., the seed for the permutation) and an index i. According to Theorem2.8, we can obtain $\sigma_v(i)$ in poly$(\log n, |v|)$ time, where σ_v is the permutation corresponding to the seed v. To find all the output bits influenced by the ith input bit, we only need to find the two endpoints of the $\sigma_v(i)$-th edge in the high-girth graph, which can be done in polylog(n) time by Theorem2.12. □

Theorem3.13. Let $\beta > 0$ be the shrinkage parameter and $\delta = \delta(n) \leq O(\log^2 n/\log\log n)$ be the error parameter. The hash family $H^{\mathrm{md}}_{\beta,\delta}$ in Construction3 is locally explicit.

Proof. Recall that the hash function H^{md} in Construction3 is obtained by the 1-detector L^{hg} in Construction2 and the reduction in Construction1. The output of the hash $H^{\mathrm{md}}_{\beta,\delta}$ consists of two parts: the output of the 1-detector L^{hg}, and several randomly sampled bits from the input. Since L^{hg} is locally explicit by Theorem3.12, it is sufficient to show that the sampling procedure is locally explicit. More precisely, given a seed v of the sampling procedure and an index i, we need to compute efficiently all the indices j such that $w_j = i$ for the vector $(w_0, \ldots, w_{b-1})$ corresponding to the seed v. This can be done using the algorithm A^{samp} from Lemma3.1. □

Theorem3.14. Let $\ell = \ell(n) = \mathrm{polylog}(n)$. The hash family H_ℓ^{expw} in Construction4 is locally explicit.

Proof. Let $S_n = \{w_0, \ldots, w_{t-1}\}$ with $t = \widetilde{O}(n^2)$ is a 0.1-biased set from Theorem2.4 and G_t is the expander in Theorem2.10. The hash function in Construction4 is defined as

$$h_v(x) \triangleq \langle w_{v_0}, x\rangle \| \langle w_{v_1}, x\rangle \| \cdots \| \langle w_{v_{\ell-1}}, x\rangle,$$

where $v = (v_0, \ldots, v_{\ell-1}) \in [t]^\ell$ is a random walk on G_t. Given the seed v (i.e., the seed of a random walk), we can produce $v_0, v_1, \ldots, v_{\ell-1}$ in $\mathrm{polylog}(n, \ell) = \mathrm{polylog}(n)$ time by Theorem2.10. Then using the algorithm A in Theorem2.4 we can check for each j whether the ith bit of w_{v_j} is 1, which indicates whether the jth output bit is influenced by the ith input bit. Enumerating all $j \in [\ell]$ gives a required $\mathrm{polylog}(n)$ time algorithm listing all the outputs influenced by a particular input bit. □

Using Theorem3.13, Theorem3.14, and Proposition3.11, we immediately obtain the explicitness of Construction5.

Corollary3.15. Let $\beta \in (0, 2]$ be a parameter and $\ell = \ell(n) \leq O(\log^2 n / \log\log n)$ be a non-decreasing function. The hash family $H_{\beta,\ell}^{\mathrm{final}}$ in Construction5 is locally explicit.

3.5 Uniformity of our construction

We have shown in Theorem3.9 that our hash function can be computed by extremely sparse $\mathsf{CC}^0[2]$ circuit or B_2 circuits with small size and depth simultaneously. It remains to clarify the uniformity of our hash construction, i.e., the complexity we need to construct the circuit given the input length and the seed. The definitions of uniformity vary with respect to the complexity measures, which can be chosen according to the applications. Typical choices include space complexity (e.g., LOGSPACE-uniform, see Section6.2.1 of [3]), time complexity (e.g., polynomial-time uniform), parallel time (see, e.g., Section3 of [29]), and descriptive complexity (see, e.g., [20]). If we choose the time complexity, for instance, we can define P-uniformity as the existence of the polynomial-time algorithm that prints the circuit given 1^n and the seed v. Indeed, it is easy to check that the $2n + o(n)$ size B_2 and $\mathsf{CC}^0[2]$ circuits for Construction5 in Theorem3.9 is P-uniform. Since our hash function can be evaluated in low-depth circuit models such as NC^1 and $\mathsf{CC}^0[2]$, the parallel time to generate the circuit also seems to be crucial for its further applications. Therefore in this section we will discuss the POLYLOGTIME-uniformity of our hash functions, defined as follows.

Definition3.16. Let C be a circuit class. A family of $\mathrm{polylog}(n)$-samplable distributions over m-output functions (e.g., hash functions or 1-detectors) $H = \{H_n\}_{n\geq 1}$ is said to be computable with POLYLOGTIME-uniform $S(n)$-size (measured in size, depth, wire complexity, etc) C-circuits if each of the functions $h_v \in \mathrm{supp}(H_n)$ corresponding to the seed v is computable by an $S(n)$-size C-circuit, supplemented with the following $\mathrm{polylog}(S(n))$-time algorithms:

- $\mathrm{SIZE}(n, v)$ returns the size of the C-circuit C_v computing h_v;

- TYPE(n, v, i) returns the type of the ith node in the circuit C_v, which indicates (1) whether it is a gate, an input variable or a constant; (2) whether it is an output node; and (3) the type (if it is a gate or a constant) or index (if it is an input variable and/or output node) of it;
- EDGE(n, v, i, j) returns the jth input of the ith node if the ith node is a gate in C_v.
- OUT(n, v, i) returns the number of the ith output node.

It is guaranteed that the nodes are numbered in topological orders, and in particular, the input variables are numbered from 1 to n.

We claim that all our results of circuit complexity for the constructions in the previous parts of this section are in fact POLYLOGTIME-uniform. It is straightforward to check the claim, so we only sketch the proof and left the details to the readers.

Theorem3.17 (Uniformity of Theorem3.9). Let $\beta \in (0, 2]$ be a parameter and $\ell = \ell(n) \leq O(\log^2 n / \log\log n)$ be a non-decreasing function. The hash function $H^{\text{final}}_{\beta,\ell}$ in Construction5 can be computed either by POLYLOGTIME-uniform depth-$(1+\lceil 2/\beta \rceil)$ $\mathsf{CC}^0[2]$ circuits of wire complexity $2n+O(n/\log\log n)$, or by POLYLOGTIME-uniform B_2 circuits of size $2n+O(n/\log\log n)$ and depth $\log n + O(\beta^{-1})$ simultaneously.

Sketch. Since the POLYLOGTIME-uniformity is close under composition, we only need to verify that Construction3 and Construction4 are POLYLOGTIME-uniform. The uniformity of Construction3 directly follows from the algorithm B in Theorem2.12, the algorithm A in Theorem2.8, and the algorithm A in Lemma3.1. The uniformity of Construction4 follows from the algorithm A in Theorem2.4 and the algorithm A in Theorem2.10. Note that the circuit for Construction4 should be slightly modified to make EDGE$(n, \cdot, \cdot)$ polylog(n)-time computable. Take the $\mathsf{CC}^0[2]$ case for example: each of the output gates has fan-in exactly n (instead of $|w_{v_i}|$ for the ith output bit, see Construction4 for the notation); the jth input wire of the ith output gate is connected to the jth input or a constant 0 according to the jth bit of w_{v_i}. □

AppendixA: Strongly explicit high-girth graphs

In this section, we will examine the construction of bipartite high-girth graphs of Lazebnik and Ustimenko [27], and verify that it is indeed strongly explicit, thereby proving Theorem2.12. Let q be an odd prime power. Let P and L be two infinite sequences of elements from $\mathbb{F}_q$ indexed as follows:

$$P = p_1, p_{1,1}, p_{1,2}, p_{2,1}, p_{2,2}, p'_{2,2}, p_{3,2}, \ldots, p_{i,i+1}, p_{i+1,i}, p_{i+1,i+1}, p'_{i+1,i+1}, p_{i+1,i+2}, \ldots,$$
$$L = l_1, l_{1,1}, l_{1,2}, l_{2,1}, l_{2,2}, l'_{2,2}, l_{3,2}, \ldots, l_{i,i+1}, l_{i+1,i}, l_{i+1,i+1}, l'_{i+1,i+1}, l_{i+1,i+2}, \ldots.$$

The names P and L mean points and lines, respectively, due to certain geometric intuition of the construction. We say P is incident with L if they satisfy the

following set of equations

$$E_1 = \begin{cases} l_{1,1} - p_{1,1} = l_1 p_1 \\ l_{1,2} - p_{1,2} = l_{1,1} p_1 \\ l_{2,1} - p_{2,1} = l_1 p_{1,1}, \end{cases} \qquad E_i = \begin{cases} l_{i,i} - p_{i,i} = l_1 p_{i-1,i} \\ l'_{i,i} - p'_{i,i} = l_{i,i-1} p_1 \\ l_{i,i+1} - p_{i,i+1} = l_{i,i} p_1 \\ l_{i+1,i} - p_{i+1,i} = l_1 p'_{i,i} \end{cases} \quad (\forall i \geq 2). \tag{3}$$

One can verify that for every point P, a line L incident with it is uniquely determined by l_1, since all other coordinates of L can be computed from the equations above iteratively. Similarly for every line L, a point P incident with it is uniquely determined by p_1. Let $P_k \in \mathbb{F}_q^k$ be the length-k prefix of P, and $L_k \in \mathbb{F}_q^k$ be the length-k prefix of L. We define $D(k,q) = (V_1, V_2, E \subseteq V_1 \times V_2)$ to be the following bipartite graph:

- $V_1 = V_2 = \mathbb{F}_q^k$;
- $(u,v) \in E \iff \exists (P, L)$, P is incident with L, $u = P_k$, and $v = L_k$.

Equivalently, u and v are connected if and only if they satisfy the first k equations of $\bigcup_i E_i$ in (3). Note that $|V_1| = |V_2| = q^k$, $|E| = q^{k+1}$, and the graph is q-regular.

TheoremA.1 (Lazebnik and Ustimenko [27]). Let $k \geq 3$ be an odd integer and q be an odd prime power. Then the girth of $D(k,q)$ is at least $k+5$.

For our purpose, it is sufficient to consider $q = p^r$ for prime $p = O(1)$. To construct the graph explicitly, we need to evaluation field operations over $\mathbb{F}_q$ for a prime power q in polylog(q) time, for which we need a representation of $\mathbb{F}_q$. Recall that $\mathbb{F}_q$ is isomorphic to $\mathbb{F}_p[x]/(Q(x))$ for any irreducible degree-r polynomial $Q \in \mathbb{F}_p[x]$. So the explicit representation of $\mathbb{F}_q$ follows from the construction of irreducible polynomials by Shoup [39].

TheoremA.2 (Shoup [39]). There is a deterministic algorithm that constructs a degree-n irreducible polynomial over $\mathbb{F}_p$ given an integer n and a prime power p in poly(n,p) time.

Now we check that $D(k,q)$ is strongly explicit, in the sense that given the index i of an edge, we can obtain the indices j_1 and j_2 in poly$(k, \log q)$ time such that the ith edge connects the j_1th vertex in V_1 and the j_2th vertex in V_2. Let $q = p^r$ for a prime $p = O(1)$. We number the vertices and edges as follows.

1. We identify the elements in the finite field $\mathbb{F}_q$ as length-r vectors from $[p]^r$, and number all the elements in the lexicographic order.

2. We identify the vertices in both V_1 and V_2 (that are length k sequences of elements in $\mathbb{F}_q$) as length-kr vectors from $[p]^{kr}$, and number them in the lexicographic order.

3. For any $i \in [q^k]$ and $j \in [q]$, the $(iq+j)$th edge connects the ith vertex P in V_1 and the unique vertex $L = (l_1, \ldots)$ in V_2 connected to P such that l_1 is the jth element in $\mathbb{F}_q$.

Under such a numbering scheme, given the index $iq + j$ of an edge with $i \in [q^k]$ and $j \in [q]$, we can easily determine its two endpoints in $\text{poly}(k, \log q)$ time. Now we are ready to prove Theorem2.12.

Remainder of Theorem2.12. Let $r = r(n) = n^{o(1)}$ be a parameter. For every sufficiently large n, there exists an $m = \Theta(n/r)$ and a regular graph $G_{m,n}$ with m vertices, n edges, and girth $\Omega(\log n/\log r)$. Moreover, there exists a polylog(n)-time algorithm $A(n, i)$ for $i \in [n]$ that outputs the indices of the two endpoints of the ith edge in $G_{m,n}$, and a polylog(n)-time algorithm $B(n, i, j)$ for $i \in [m]$ that outputs the jth edge attaching to the ith vertex.

Proof. Let q be the power of 3 in the interval $[r, 3r)$. Applying TheoremA.2, we construct a fixed representation of $\mathbb{F}_q$ in polylog(q) time so that the field operations over $\mathbb{F}_q$ can be evaluated in polylog(q) time. Let $k \triangleq \lfloor \log_q n \rfloor - 1$ (i.e., k is the largest integer such that $q^{k+1} \le n$), $\ell \triangleq \lceil n/q^{k+1} \rceil$, and $m \triangleq 2q^k\ell$. Note that

$$m \le 2q^k \left(\frac{n}{q^{k+1}} + 1 \right) = \frac{2n}{q} + \frac{2q^{k+1}}{q} \le O(n/r),$$
$$m \ge 2q^k \cdot \frac{n}{q^{k+1}} \ge \Omega(n/r),$$

and therefore $m = \Theta(n/r)$. Now we construct a graph $G_{m,n}$ with ℓ connected components as follows. Each of the first $\ell - 1$ connected components is a copy of $D(q, k)$ with $2q^k$ vertices and q^{k+1} edges. The last connected component is a subgraph of $D(q, k)$ with $2q^k$ vertices but only the first $n - q^{k+1}(\ell - 1)$ edges. By TheoremA.1, it is easy to see that the girth of $G_{m,n}$ is at least $k = \Omega(\log n/\log q) = \Omega(\log n/\log r)$. According to our numbering scheme and related discussions, it is also easy to verify that given an edge index i, we can compute the two endpoints of the ith edge in $G_{m,n}$ in polylog(n) time. Furthermore, the jth edge attaching to the ith vertex can be easily computed since each of the q edges attaching to the ith vertex is uniquely determined by $l_1 \in \mathbb{F}_q$ or $p_1 \in \mathbb{F}_q$ (see Equation3). □

4 Sharp bootstrapping results from hash functions

We now show the extremely sharp bootstrapping results for small linear size circuits based on the almost universal hash function constructed above, using a refined kernelization method of Chen, Jin, and Williams [9,10]. We will first prove the general hardness magnification theorem for all sparse NP languages in Section4.1. In Section4.2, we will present stronger hardness magnification results for MCSP, which will utilize the explicitness of our hash construction. Then we will show in Section4.3 that similar techniques can be applied to obtain a bootstrapping result for explicit obstructions (see, e.g., [10]), which formalizes the explicit proofs of circuit lower bounds. In Section4.4, we construct explicit obstructions and prove circuit lower bounds that tightly match these bootstrapping results.

4.1 Hardness magnification for all sparse NP languages

We first prove the most general version of the hardness magnification result. More discussions are presented after the proof.

Theorem4.1. Let $s = s(n)$ and $T = T(n)$ be two functions such that

- $\omega(\log n) \leq s(n) \leq O(\log^2 n/\log\log n)$, $s(\Theta(n/\log^2 n)) = \Theta(s(n))$, s is non-decreasing;
- $n^\gamma \leq T(n) \leq 2^{O(n)}$, where $\gamma > 1$ is an absolute large constant.

Then, if there is a $2^{s(n)}$-sparse language L in $\mathsf{NTIME}[T(n)]$, such that L cannot be computed by probabilistic circuits of size $2n + O(ns/\log^2 n)$ within error $\exp(-\Omega(s))$, it then follows that $\mathsf{NTIME}[T(2^{O(n^{1/5})})] \not\subseteq \mathsf{SIZE}[2^{cn^{1/5}}]$ for all $c > 1$.

Proof. Towards a contradiction we assume that $\mathsf{NTIME}[T(2^{O(n^{1/5})})] \subseteq \mathsf{SIZE}[2^{cn^{1/5}}]$ for some $c > 1$. Let L be a $2^{s(n)}$-sparse language in $\mathsf{NTIME}[T(n)]$. We now show that L can be computed by probabilistic circuits of size $2n+O(ns/\log^2 n)$ within error $\exp(-\Omega(s))$. Let $H = H^{\text{final}}_{\beta,\ell} = \{H_n\}_{n\geq 1}$ be the linear $O(\ell \log^{1+\beta} n)$-samplable $\Theta(\ell)$-output $\exp(-\Omega(\ell))$-almost universal hash function in Theorem3.9 with $\ell(n) = O(s)$ and $\beta = 2$, such that the collision probability is at most 2^{-2s}.[2] The circuit complexity of each $h \in \text{supp}(H_n)$ is bounded by $2n + O(ns/\log^2 n)$. Since H is efficiently samplable, there is a polynomial time algorithm $M(1^n, v, \cdot)$ computing the hash function corresponding to the seed v, where $|v| = O(\ell \log^3 n) = O(s \log^3 n)$.[3] We now define an intermediate language $L' = \{(n, v, h) \mid \exists x \in \{0,1\}^n \cap L, M(1^n, v, x) = h\}$. For any sufficiently large n, we pad the language to make the corresponding length of the tuple in L' has length exactly $m = m(n) = \lfloor(\log(n)/(2c))^5\rfloor$ to obtain L''. That is, for sufficiently large n and v, h of appropriate length, we define

$$z_{n,v,h} \triangleq (n, v, h)\|1\|0^{m(n)-|(n,v,h)|-1}$$

and we have

$$z_{n,v,h} \in L'' \iff (n, v, h) \in L'.$$

Using the straightforward non-deterministic algorithm for L' (guessing $x \in \{0,1\}^n$ and determine whether $M(1^n, v, x) = h$), we can show that $L'' \in \mathsf{NTIME}[T(2^{O(m^{1/5})})] \subseteq \mathsf{SIZE}[2^{cm^{1/5}}]$, the second containment follows from our assumption. The key in our argument is an algorithm for the sparse language L with an oracle access to this intermediate problem L'', and then replace the oracle with the small size circuit by assumption. This is formalized in the following lemma.

Lemma4.2. There is a uniform family $D = \{D_n\}_{n\geq 1}$ of probabilistic oracle circuits of size $2n + O(ns/\log^2 n)$, such that for every input length n, the followings hold:

1. Every $D \in \text{supp}(D_n)$ contains at most one L'' oracle gate of fan-in $\lfloor \log^5(n)/(32c^5)\rfloor$.

[2] In particular, assume that α is a constant such that H is $\exp(-\alpha\ell)$-almost universal hash function, then we take $\ell(n) = \lceil 2s/\alpha\rceil$.

[3] That is, for every $x \in \{0,1\}^n$ and every seed v, we have $M(1^n, v, x) = h_v(x)$, where h_v is the hash function corresponding to the seed v.

2. D decides the language L with error at most 2^{-s}.

Proof. Consider the following probabilistic circuit family $D = \{D_n\}_{n\geq 1}$. Given input x of length n, D_n is constructed as follows: we randomly choose a seed v to sample a hash function h_v from H_n; and then query whether $z_{n,v,h_v(x)}$ is in L'' via the oracle gate, where h_v is the hash function corresponding to the seed v. The circuit complexity of D_n equals the circuit complexity of the hash function h_v, which is at most $2n+O(ns/\log^2 n)$ by Theorem3.9, and every $D \in \mathrm{supp}(D_n)$ only needs to call the L'' oracle once with input length $m = \lfloor \log^5(n)/(32c^5) \rfloor$. Clearly, by the definition of L'', for any $x \in L$, D_n accepts x with probability 1. For any $x \notin L$, we have

$$\begin{aligned}\Pr_v[D_n \text{ accepts } x] &= \Pr_{h\leftarrow H_n}[\exists x' \in L \cap \{0,1\}^n \text{ s.t. } h(x) = h(x')] \\ &\leq \sum_{x'\in L\cap\{0,1\}^n} \Pr_{h\leftarrow H_n}[h(x) = h(x')] \\ &= 2^s \cdot 2^{-2s} \leq 2^{-s}.\end{aligned}$$

□

Now applying Lemma4.2 and replacing the oracle query by circuits using the fact that $L'' \in \mathsf{SIZE}[2^{cn^{1/5}}]$, we finish the proof of Theorem4.1. □

We now discuss some typical choices of parameters in the above theorem. Let the sparsity parameter $s(n) = \log^2 n/\log\log n$.[4]

- A standard form of hardness magnification result similar to the one in [9] but quantitatively stronger can be obtained by choosing $T(n) = \mathrm{poly}(n)$. Then the theorem says that, if there exists an $n^{\log n/\log\log n}$-sparse language in NP that does not have probabilistic circuits of size $2n+O(n/\log\log n)$, then $\mathsf{NTIME}[2^{O(n^{1/5})}] \not\subseteq \mathsf{SIZE}[2^{cn^{1/5}}]$ for all $c > 0$. By a padding argument, this implies that $\mathsf{NP} \not\subseteq \mathsf{SIZE}[n^c]$ for all $c > 0$. This establishes Theorem1.2.

- Being less ambitious, we can let $T(n) = 2^{n^{o(1)}}$ and show that, even the existence of an $n^{\log n/\log\log n}$-sparse language in $\mathsf{NTIME}[2^{n^{o(1)}}]$ that does not have probabilistic circuits of size $2n+O(n/\log\log n)$, would be enough to imply that $\mathsf{NTIME}[2^{2^{o(n^{1/5})}}] \not\subseteq \mathsf{SIZE}[2^{cn^{1/5}}]$, and hence $\mathsf{NTIME}[2^{n^{o(1)}}] \not\subseteq \mathsf{SIZE}[n^c]$ for all $c > 0$ and $\mathsf{NEXP} \not\subseteq \mathsf{P/poly}$, which would already be a major breakthrough in circuit complexity. This establishes Theorem1.3.

4.2 Hardness magnification for MCSP

We now utilized the strongly explicit hash function constructed above to obtain a stronger hardness magnification theorem for MCSP. Intuitively, this is possible since the yes-instances of MCSP can be efficiently encoded in a much shorter string. Indeed, for MCSP$[s(n)]$, one only need $O(s(n)\log s(n))$ bits to encode a small circuit, instead of 2^n bits for a whole truth table. We assume a paddable encoding of circuits, i.e., any string x and $x\|0$ encode the same circuit. This

[4]For a typical $2^{s(n)}$-sparse language in NP, consider MCSP$[n^{1.9}]$ on input length $N = 2^n$.

can be done with only a constant overhead. For a circuit C, we use the notation $\langle C \rangle$ to denote its encoding.

Theorem4.3. Let $n \leq s(n) \leq O(n^2/\log^2 n)$ be a non-decreasing size parameter that satisfies $s(\log(\Theta(n/\log^2 n))) = \Theta(s(\log n))$. Let $N = 2^n$ be the truth table length. Let $g = g(n) = s(n)\log s(n)$. If MCSP$[s(n)]$ cannot be computed by probabilistic circuits of size $2N + O(Ng/\log^2 N)$ within error $\exp(-\Omega(g))$, then there is some $c \in (0,1)$ such that $\oplus\mathsf{P} \not\subseteq \mathsf{SIZE}[2^{N^c}]$.

Intuition. The proof is similar to the one for Theorem4.1, but we need to make the oracle from Lemma3.1 computable in $\oplus\mathsf{P}$ instead of in super-polynomial non-deterministic time. Concretely, we need to slightly modify the intermediate problem so that we can make use of the strongly explicitness of our hash construction H^{final}.

Proof. Towards a contradiction, suppose that $\oplus\mathsf{P} \subseteq \mathsf{SIZE}[2^{N^c}]$ for all $c \in (0,1)$, we now prove that MCSP$[s(n)]$ can be computed by probabilistic circuits of size $2N + O(Ng/\log^2 N)$ within error $2^{-\Omega(g(n))}$. Let $N = 2^n$ be the truth table length of n-input functions. Take $H = H^{\mathrm{final}}_{\beta,\ell} = \{H_N\}_{N\geq 1}$ to be the linear $O(\ell \log^{1+\beta} N)$-samplable $\Theta(\ell)$-output $\exp(-\Omega(\ell))$-almost universal hash function in Theorem3.9 with $\ell(N) = O(g(n)) = O(g(\log N))$ and $\beta = 2$, such that the collision probability is at most 2^{-2g}.[5] We denote the hash function of input length N corresponding to the seed v as $H_{N,v}(\cdot)$. By the strongly explicitness (Corollary3.15), there exists a algorithm $M(N, v, i, j)$ running in polylog$(N) =$ poly(n) time that decides whether the ith output bit depends on the jth input bit in $H_{N,v}$. Note that for a circuit C of input length n,[6]

$$H_{N,v}(\mathrm{tt}(C))_i = \sum_{j=0}^{N-1} M(N, v, i, j) \cdot C(j) \pmod 2,$$

and $M(N, v, i, j) \cdot C(j)$ can be computed in poly$(n, |\langle C \rangle|)$ time, so the language $L^h = \{(N, i, v, \langle C \rangle) \mid H_{N,v}(\mathrm{tt}(C))_i = 1\}$ is decidable in $\oplus\mathsf{P}$. Hence if we define an intermediate language $L' = \{(N, v, h) \mid \exists C \in \mathsf{SIZE}[s(n)], H_{N,v}(\mathrm{tt}(C)) = h\}$ similar to the proof of Theorem4.1, it is now decidable in NP given oracle access to $\oplus\mathsf{P}$. Let $m = O(\ell \log^3 n)$ be the input length of L'. Note that $L' \in \mathsf{NP}^{\oplus\mathsf{P}} \subseteq \mathsf{BPP}^{\oplus\mathsf{P}} \subseteq \mathsf{P}^{\oplus\mathsf{P}}_{/\mathrm{poly}} \subseteq \oplus\mathsf{P}/\mathrm{poly}$, where $\mathsf{NP}^{\oplus\mathsf{P}} \subseteq \mathsf{BPP}^{\oplus\mathsf{P}}$ follows from [42] (see also [15]) and $\mathsf{P}^{\oplus\mathsf{P}}_{/\mathrm{poly}} \subseteq \oplus\mathsf{P}/\mathrm{poly}$ follows from $\oplus\mathsf{P}^{\oplus\mathsf{P}} \subseteq \oplus\mathsf{P}$ [37].[7] Since $\oplus\mathsf{P} \subseteq [2^{N^c}]$ for all $c \in (0,1)$ by the assumption, we know that the evaluation of $\oplus\mathsf{P}/\mathrm{poly}$ circuits can be computable in $\mathsf{SIZE}[2^{N^c}]$ for all $c \in (0,1)$, which implies that $L' \in \mathsf{SIZE}[\sqrt{N}]$. The rest of the proof follows similar to Theorem4.1 and the fact that the $m = O(\ell \log^3 N) = o(\log^5 N)$. □

[5] Note that since $s(n) \leq O(n^2/\log^2 n)$, we have $\ell(N) = O(\log^2 N/\log\log N)$ and ℓ is non-decreasing. Hence, ℓ satisfies the requirements in Theorem3.9.

[6] Here we use the notation $\mathrm{tt}(C)$ to represent the truth table of C. Particularly, if C is a single-output circuit of n inputs, then $\mathrm{tt}(C)$ is a string of length 2^n such that $\mathrm{tt}(C)_i = C(i)$.

[7] Since $\oplus\mathsf{P}^{\oplus\mathsf{P}} \subseteq \oplus\mathsf{P}$ implies that $\mathsf{P}^{\oplus\mathsf{P}} \subseteq \oplus\mathsf{P}$, it follows that the evaluation of polynomial-size circuits with $\oplus\mathsf{P}$ oracles can be computable in $\oplus\mathsf{P}$, which further implies that $\mathsf{P}^{\oplus\mathsf{P}}_{/\mathrm{poly}} \subseteq \oplus\mathsf{P}/\mathrm{poly}$.

4.3 Explicit obstruction

We now formally state and prove our results regarding explicit obstructions. We begin by formally defining the notion of explicit obstructions.

Definition4.4 (Explicit obstruction). An explicit obstruction of size $S(n)$ computable in C against D is a family of lists of input-output pairs $O = \{O_n\}_{n\geq 1}$ satisfying the following.

- $|O_n| \leq S(n)$ for all sufficiently large n.
- There is a machine in C that prints the set O_n given input 1^n.
- For every n, $O_n = \{(x_i, y_i)\}$ satisfies that $x_i \neq x_j$ for all $i \neq j$.
- For all sufficiently large n, and for every n-input D circuit f, there is a pair $(x_i, y_i) \in O_n$ such that $f(x_i) \neq y_i$.

Theorem4.5. There is an absolute constant $\gamma > 0$ such that the following holds. Let $n^\gamma \leq T(n) \leq 2^n$ and $\log n \leq s(n) \leq \min\{O(\log^2 n/\log\log n), \log T(n)\}$ being non-decreasing and satisfying $s(\Theta(n/\log^2 n)) = \Theta(s(n))$. If there is an explicit obstruction of size $2^{s(n)}$ computable in $\mathsf{DTIME}[T(n)]$ against $2n + O(n/\log\log n)$-size circuits, then $\mathsf{DTIME}[T(2^{O(n^{1/5})})] \not\subseteq \mathsf{SIZE}[2^{cn^{1/5}}]$ for all $c > 1$.

Proof. Let $t(n) = 2^{s(n)}$. Towards a contradiction we assume that there is some constant $c > 1$, such that $\mathsf{DTIME}[T(2^{O(n^{1/5})})] \subseteq \mathsf{SIZE}[2^{cn^{1/5}}]$. Suppose that $O = \{O_n\}_{n\geq 1}$ is an explicit obstruction against $2n + O(n/\log\log n)$ size circuits. For any sufficiently large input length n, we suppose that $O_n = \{(x_{n,1}, y_{n,1}), (x_{n,2}, y_{n,2}), \ldots, (x_{n,t(n)}, y_{n,t(n)})\}$. Our goal is to design a circuit with extremely small size, but agree with O_n on all its input-output pairs. Following the similar proof outline as for Theorem4.1, we begin by using an almost universal hash function to kernalize the inputs from O_n. Let $H = H^{\text{final}}_{\ell,\beta} = \{H_n\}_{n\geq 1}$ be the linear $O(\ell \log^{1+\beta} n)$-samplable $\Theta(\ell)$-output $\exp(-\Omega(\ell))$-almost universal hash function in Theorem3.9 with $\ell(n) = O(s)$ and $\beta = 2$ such that the collision probability is at most 2^{-3s}. Let n be a sufficiently large input length. We call a hash function h good if it is perfect on the inputs in O_n, i.e., any two distinct inputs in O_n have different hash values assigned by h. For a randomly chosen function $h \leftarrow H_n$, the probability of it not being good is bounded by

$$\begin{aligned}\Pr_{h\leftarrow H_n}[\exists 1 \leq i < j \leq t(n) \text{ s.t. } h(x_{n,i}) = h(x_{n,j})] &\leq \sum_{1\leq i<j\leq t(n)} \Pr_{h\leftarrow H_n}[h(x_{n,i}) = h(x_{n,j})] \\ &\leq \binom{2^s}{2} 2^{-3s} \leq 2^{-s}.\end{aligned}$$

So a good hash function always exists in the support of H_n for all sufficiently large n. For any large input length n, we arbitrarily fix such a good hash and denote its seed by v_n^{good}. Let h_v be the hash function corresponding to the seed v. We define an intermediate language $L' = \{(n, v, h) \mid \exists 1 \leq i \leq$

$t(n), y_{n,i} = 1 \wedge h_v(x_{n,i}) = h\}$. We again pad its input to have length exactly $m = \lfloor(\log(n)/(2c))^5\rfloor$, and form a padded language L''. Then $L'' \in \mathsf{DTIME}[T(2^{O(m^{1/5})})] \subseteq \mathsf{SIZE}[2^{cm^{1/5}}]$. We also note that $2^{cm^{1/5}} \leq \sqrt{n}$. Then we consider the function $f_n(x) \triangleq L'(n, v_n^{\text{good}}, H_n(v_n^{\text{good}}, x))$, where $H_n(v, \cdot)$ is the hash function in H_n corresponding to the seed v. By an argument similar to that of Theorem4.1, we can show that f_n can be decided by a circuit of size $2n + O(n/\log\log n) + \sqrt{n} = 2n + O(n/\log\log n)$, but totally agrees with O_n, contradicting to the assumption that O is an explicit obstruction against circuits of such size. □

4.4 Unconditional lower bounds for sparse languages

Now we complement the results mentioned in previous subsections by constructing an explicit obstruction against B_2 circuits of size $2n - O(1)$ and a corresponding sparse language in P with a $2n-O(1)$ probabilistic circuit lower bound. They form sharp bootstrapping thresholds together with Theorem4.1 and Theorem4.5. The main idea behind our explicit obstruction is the investigation of a combinatorial structure in Boolean circuits called critical path introduced in [13], which was used to prove $2n-O(1)$ circuit lower bounds for PRFs and hash functions. For the simplicity of presentation, we assume without loss of generality that our circuits are normalized, in the sense that there is no non-output gates with out-degree 0. This is without loss of generality since redundant gates with out-degree 0 can be removed.

Definition4.6 (Critical path). Let C be a circuit, and u be an input variable of it. The critical path of u in C is a sequence of vertices $v_0, v_1, \ldots, v_k$ satisfying the following conditions:

1. $v_0 = u$, and v_i is a descendent of v_{i-1} for all $i \geq 1$, and
2. out-degree$(v_i) = 1$ for all $0 \leq i < k$, and out-degree$(v_k) \neq 1$.

Fix a circuit with n input bits, we can obtain a total of n critical paths. The crucial observation in [13] is that, if all critical paths do not intersect with one another, then the circuit must contain at least $2n - O(1)$ gates. This is formalized below.

Lemma4.7 ([13], Lemma6.4). For any normalized n-input single-output circuit C with no intersecting critical paths and no input variables with out-degree 0, the number of gates in the circuit is at least $2n - 2$.

Let $O = \{O_n \subseteq \{0,1\}^n \times \{0,1\}\}_{n\geq 1}$, where

$$O_n = \{(x, 0) \mid |x| \in \{0, 2, n-2, n-1\}\} \cup \{(x, 1) \mid |x| \in \{1, n\}\}.$$

In the remaining part of the section, we will first prove that O is an explicit obstruction against $2n - 2$ size circuits, and then present a general connection between explicit obstruction and probabilistic circuit lower bounds.

Explicit obstruction. According to Lemma4.7, we only need to prove that any circuit with intersecting critical paths or input variables with out-degree 0 does not fully agree with O. For circuits with input variables of out-degree 0,

this can be verified straightforwardly. Now we prove the case for circuits with intersecting critical paths.

Lemma4.8. For any circuit $C_n : \{0,1\}^n \to \{0,1\}$ with intersecting critical paths, there must exist a pair $(x, b) \in O_n$ such that $C_n(x) \neq b$.

Proof. Suppose that the critical paths of u and v in C_n intersect. Let G be the first gate on the intersection. Let f_G be the function computed by G. Assume that we take a restriction ρ to all variables except u and v, and consider the restricted function $C_n \restriction \rho$. The key observation is that, for any ρ, there are functions $\phi_\rho, \psi_\rho, \chi_\rho : \{0,1\} \to \{0,1\}$ such that

$$C_n \restriction \rho(u, v) = \chi_\rho(f_G(\phi_\rho(u), \psi_\rho(v))).$$

Based on the characterization of different functions in B_2, we call a function f quadratic if it has the form $f(u, v) = ((u \oplus c_1) \wedge (v \oplus c_2)) \oplus c_3$. We call a function linear if it has the form $f(u, v) = u \oplus v \oplus c$. The pivotal point in [13] is that for a fixed pair of u and v, $C_n \restriction \rho$ cannot be quadratic under some restriction ρ_1, then become linear under another restriction ρ_2. However, by checking the truth table it can be verified that our construction of O_n forces $C_n \restriction \rho$ to be the linear function $(u, v) \mapsto u \oplus v$ for all-zero restriction, and the quadratic function $(u, v) \mapsto u \wedge v$ for all-one restriction, which leads to contradiction. □

Corollary4.9. O is an explicit obstruction against $2n - 2$ size B_2 circuits.

Probabilistic circuit lower bound. We now present a general reduction from explicit obstruction to probabilistic circuit lower bounds for sparse languages in P.

Lemma4.10. For any circuit class C, if there exists an explicit obstruction O of size $S(n)$ against C, then every language agreeing with O cannot be computed by probabilistic C-circuits with error probability less than $1/S(n)$, even infinitely often.

Proof. Towards contradiction, let L be a language agreeing with O, and assume that there exists a probabilistic C-circuit C computing L with error probability smaller than $1/S(n)$, for infinitely many n. Then for those n, we have

$$\mathbb{E}_C[|\{(x, b) \in O_n \mid C(x) \neq b|\}] < 1.$$

By the averaging argument, there exists a deterministic circuit that agrees with all pairs in O_n. This guarantees the existence of a family of circuit in C that agrees with O on those n, which leads to a contradiction. □

In particular, let $L = \{L_n\}_{n \geq 1}$ be a language such that $L_n \triangleq \{x \in \{0,1\}^n \mid (x, 1) \in O_n\}$, then it is a sparse language in P that agrees with O. Together with Corollary4.9, we obtain the following lower bound.

Corollary4.11. There exists a $O(n)$-sparse language in P that cannot be computed (even infinitely often) by probabilistic B_2 circuits of size $2n - 2$ with error probability smaller than $1/(2n^2)$.

Remark4.12. We note that the error probability in the lower bound cannot be trivially boosted to 1/3, since the complexity overhead of error reduction is not affordable when we are dealing with small linear-size circuits. Therefore it might be significantly more difficult to prove a similar circuit lower bound with constant error probability.

References

[1] Noga Alon, Oded Goldreich, Johan Håstad, and René Peralta. Simple constructions of almost k-wise independent random variables. In 31st Annual Symposium on Foundations of Computer Science, St. Louis, Missouri, USA, October 22-24, 1990, Volume II, pages 544–553. IEEE Computer Society, 1990. doi:10.1109/FSCS.1990.89575.

[2] Benny Applebaum and Pavel Raykov. Fast pseudorandom functions based on expander graphs. In Theory of Cryptography - 14th International Conference, TCC 2016-B, Beijing, China, October 31 - November 3, 2016, Proceedings, Part I, volume 9985 of Lecture Notes in Computer Science, pages 27–56, 2016. doi:10.1007/978-3-662-53641-4_2.

[3] Sanjeev Arora and Boaz Barak. Computational Complexity - A Modern Approach. Cambridge University Press, 2009.

[4] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings, volume 7237 of Lecture Notes in Computer Science, pages 719–737. Springer, 2012. doi:10.1007/978-3-642-29011-4_42.

[5] Andrej Bogdanov and Alon Rosen. Pseudorandom functions: Three decades later. In Tutorials on the Foundations of Cryptography, pages 79–158. Springer International Publishing, 2017. doi:10.1007/978-3-319-57048-8_3.

[6] Larry Carter and Mark N. Wegman. Universal classes of hash functions. J. Comput. Syst. Sci., 18(2):143–154, 1979. doi:10.1016/0022-0000(79)90044-8.

[7] L. Sunil Chandran. A high girth graph construction. SIAM J. Discret. Math., 16(3):366–370, 2003. doi:10.1137/S0895480101387893.

[8] Lijie Chen, Shuichi Hirahara, Igor Carboni Oliveira, Ján Pich, Ninad Rajgopal, and Rahul Santhanam. Beyond natural proofs: Hardness magnification and locality. In 11th Innovations in Theoretical Computer Science Conference, ITCS 2020, January 12-14, 2020, Seattle, Washington, USA, volume 151 of LIPIcs, pages 70:1–70:48. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPIcs.ITCS.2020.70.

[9] Lijie Chen, Ce Jin, and R. Ryan Williams. Hardness magnification for all sparse NP languages. In 60th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2019, Baltimore, Maryland, USA,

November 9-12, 2019, pages 1240–1255. IEEE Computer Society, 2019. doi:10.1109/FOCS.2019.00077.

[10] Lijie Chen, Ce Jin, and R. Ryan Williams. Sharp threshold results for computational complexity. In Proccedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, Chicago, IL, USA, June 22-26, 2020, pages 1335–1348. ACM, 2020. doi:10.1145/3357713.3384283.

[11] Ruiwen Chen and Valentine Kabanets. Correlation bounds and #SAT algorithms for small linear-size circuits. Theor. Comput. Sci., 654:2–10, 2016. doi:10.1016/j.tcs.2016.05.005.

[12] Evgeny Demenkov and Alexander S. Kulikov. An elementary proof of a 3n - o(n) lower bound on the circuit complexity of affine dispersers. In Mathematical Foundations of Computer Science 2011 - 36th International Symposium, MFCS 2011, Warsaw, Poland, August 22-26, 2011. Proceedings, volume 6907 of Lecture Notes in Computer Science, pages 256–265. Springer, 2011. doi:10.1007/978-3-642-22993-0_25.

[13] Zhiyuan Fan, Jiatu Li, and Tianqi Yang. The exact complexity of pseudorandom functions and the black-box natural proof barrier for bootstrapping results in computational complexity. Electron. Colloquium Comput. Complex., page 125, 2021. To appear in STOC 2022. URL: https://eccc.weizmann.ac.il/report/2021/125.

[14] Magnus Gausdal Find, Alexander Golovnev, Edward A. Hirsch, and Alexander S. Kulikov. A better-than-3n lower bound for the circuit complexity of an explicit function. In IEEE 57th Annual Symposium on Foundations of Computer Science, FOCS 2016, 9-11 October 2016, Hyatt Regency, New Brunswick, New Jersey, USA, pages 89–98. IEEE Computer Society, 2016. doi:10.1109/FOCS.2016.19.

[15] Lance Fortnow. A simple proof of Toda's theorem. Theory Comput., 5(1):135–140, 2009. doi:10.4086/toc.2009.v005a007.

[16] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. J. ACM, 33(4):792–807, 1986. doi:10.1145/6490.6503.

[17] Alexander Golovnev, Edward A. Hirsch, Alexander Knop, and Alexander S. Kulikov. On the limits of gate elimination. J. Comput. Syst. Sci., 96:107–119, 2018. doi:10.1016/j.jcss. 2018.04.005.

[18] Johan Håstad. The shrinkage exponent of de Morgan formulas is 2. SIAM J. Comput., 27(1):48–64, 1998. doi:10.1137/S0097539794261556.

[19] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. SIAM J. Comput., 28(4):1364–1396, 1999. doi: 10.1137/S0097539793244708.

[20] William Hesse, Eric Allender, and David A. Mix Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. J. Comput. Syst. Sci., 65(4):695–716, 2002. doi:10.1016/S0022-0000(02)00025-9.

[21] Shuichi Hirahara and Rahul Santhanam. On the average-case complexity of MCSP and its variants. In 32nd Computational Complexity Conference, CCC 2017, July 6-9, 2017, Riga, Latvia, volume 79 of LIPIcs, pages 7:1–7:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. doi:10.4230/LIPIcs.CCC.2017.7.

[22] Russell Impagliazzo, Raghu Meka, and David Zuckerman. Pseudorandomness from shrinkage. J. ACM, 66(2):11:1–11:16, 2019. doi:10.1145/3230630.

[23] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Cryptography with constant computational overhead. In Proceedings of the 40th Annual ACM Symposium on Theory of Computing, Victoria, British Columbia, Canada, May 17-20, 2008, pages 433–442. ACM, 2008. doi:10.1145/1374376.1374438.

[24] Kazuo Iwama and Hiroki Morizumi. An explicit lower bound of 5n - o(n) for boolean circuits. In Mathematical Foundations of Computer Science 2002, 27th International Symposium, MFCS 2002, Warsaw, Poland, August 26-30, 2002, Proceedings, volume 2420 of Lecture Notes in Computer Science, pages 353–364. Springer, 2002. doi:10.1007/3-540-45687-2_29.

[25] Eyal Kaplan, Moni Naor, and Omer Reingold. Derandomized constructions of k-wise (almost) independent permutations. Algorithmica, 55(1):113–133, 2009. doi:10.1007/ s00453-008-9267-y.

[26] Oded Lachish and Ran Raz. Explicit lower bound of 4.5n - o(n) for boolena circuits. In Jeffrey Scott Vitter, Paul G. Spirakis, and Mihalis Yannakakis, editors, Proceedings on 33rd Annual ACM Symposium on Theory of Computing, July 6-8, 2001, Heraklion, Crete, Greece, pages 399–408. ACM, 2001. doi:10.1145/380752.380832.

[27] Felix Lazebnik and Vasiliy A. Ustimenko. Explicit construction of graphs with an arbitrary large girth and of large size. Discret. Appl. Math., 60(1-3):275–284, 1995. doi:10.1016/ 0166-218X(94)00058-L.

[28] Jiatu Li and Tianqi Yang. 3.1n - o(n) circuit lower bounds for explicit functions. Electron. Colloqu ium Comput. Complex., page 23, 2021. To appear in STOC 2022. URL: https: //eccc.weizmann.ac.il/report/2021/023.

[29] Richard J. Lipton and Ryan Williams. Amplifying circuit lower bounds against polynomial time, with applications. Comput. Complex., 22(2):311–343, 2013. doi:10.1007/s00037-013-0069-5.

[30] Dylan M. McKay, Cody D. Murray, and R. Ryan Williams. Weak lower bounds on resource- bounded compression imply strong separations of complexity classes. In Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Phoenix, AZ, USA, June 23-26, 2019, pages 1215–1225. ACM, 2019. doi:10.1145/3313276.3316396.

[31] Eric Miles and Emanuele Viola. Substitution-permutation networks, pseudorandom functions, and natural proofs. J. ACM, 62(6):46:1–46:29, 2015. doi:10.1145/2792978.

[32] Ketan Mulmuley. On P vs. NP and geometric complexity theory: Dedicated to sri ramakrishna. J. ACM, 58(2):5:1–5:26, 2011. doi:10.1145/1944345.1944346.

[33] Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. J. ACM, 51(2):231–262, 2004. doi:10.1145/972639.972643.

[34] Igor Carboni Oliveira, Ján Pich, and Rahul Santhanam. Hardness magnification near state-of- the-art lower bounds. Theory Comput., 17:1–38, 2021.

[35] Igor Carboni Oliveira and Rahul Santhanam. Hardness magnification for natural problems. In 59th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2018, pages 65–76, 2018. doi:10.1109/FOCS.2018.00016.

[36] Anna Pagh and Rasmus Pagh. Uniform hashing in constant time and optimal space. SIAM J. Comput., 38(1):85–96, 2008. doi:10.1137/060658400.

[37] Christos H. Papadimitriou and Stathis Zachos. Two remarks on the power of counting. In Theoretical Computer Science, 6th GI-Conference, Dortmund, Germany, January 5-7, 1983, Proceedings, volume 145 of Lecture Notes in Computer Science, pages 269–276. Springer, 1983. doi:10.1007/BFb0009651.

[38] Alexander A. Razborov and Steven Rudich. Natural proofs. J. Comput. Syst. Sci., 55(1):24–35, 1997. doi:10.1006/jcss.1997.1494.

[39] Victor Shoup. New algorithms for finding irreducible polynomials over finite fields. In 29th Annual Symposium on Foundations of Computer Science, White Plains, New York, USA, 24-26 October 1988, pages 283–290. IEEE Computer Society, 1988. doi:10.1109/SFCS.1988.21944.

[40] Daniel A. Spielman. Linear-time encodable and decodable error-correcting codes. IEEE Trans. Inf. Theory, 42(6):1723–1731, 1996. doi:10.1109/18.556668.

[41] Avishay Tal. Shrinkage of de Morgan formulae by spectral techniques. In 55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014, pages 551–560. IEEE Computer Society, 2014. doi:10.1109/FOCS.2014.65.

[42] Leslie G. Valiant and Vijay V. Vazirani. NP is as easy as detecting unique solutions. Theor. Comput. Sci., 47(3):85–93, 1986. doi:10.1016/0304-3975(86)90135-0.

[43] Emanuele Viola. The communication complexity of addition. Comb., 35(6):703–747, 2015. doi:10.1007/s00493-014-3078-3.